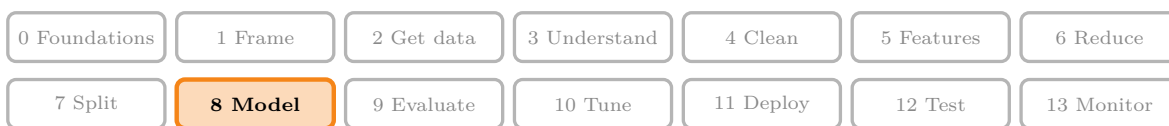


Vanishing and Exploding Gradients

Contents

1. Overview	2
2. Prerequisites	3
3. The vanishing gradient problem	3
3.1 Multiplying small numbers	3
3.2 Where the small factors come from	3
3.3 What a tiny gradient does to the update	4
4. Seeing it in Keras	4
4.1 The experiment	4
4.2 The gradient of each layer	5
4.3 The weights do not move, the loss does not fall	5
5. How to recognise it	6
6. Five ways to fix it	6
6.1 Reduce the depth	7
6.2 Use ReLU	7
6.3 Initialise the weights properly	8
6.4 Batch normalisation	8
6.5 Residual networks	8
7. The exploding gradient problem	8
7.1 Multiplying large numbers	8
7.2 An exploding network in Keras	9
7.3 Gradient clipping	9
8. Summary	10
9. Key terms	10

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



- **Builds on:** Training curves (History) ([Note 1013](#)); Backpropagation ([Note 1017](#)).
- **Leads to:** Backpropagation ([Note 1019](#)); Improving a neural network ([Note 1021](#)).

1. Overview

Key point: Backpropagation multiplies many factors together to get the gradient of an early layer. If they are mostly below 1 (as with sigmoid), the gradient vanishes and the early layers stop learning; if they are mostly above 1, it explodes and training breaks.

In backpropagation every weight receives an update proportional to the derivative of the loss with respect to it. In a deep network, the derivative for a weight near the input is a long product, with one factor per layer between it and the loss. Two things can go wrong:

- **Vanishing gradient problem:** the factors are small, the product is vanishingly small, and the weight hardly changes. In the worst case the network stops training.
- **Exploding gradient problem:** the factors are large, the product is huge, and the updates throw the weights around at random.

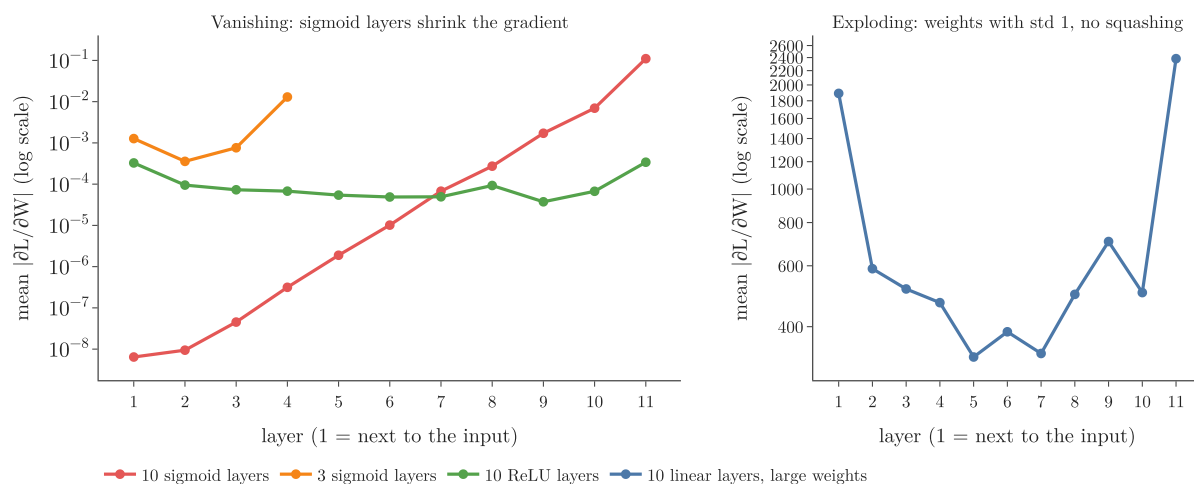


Figure 1: Mean size of the gradient of each layer's weights at the start of training. Left: 10 sigmoid layers (red) shrink it about ten-million-fold towards the input. Right: large weights without squashing make every gradient huge

Figure 1 shows both on real networks. This Note explains why it happens, how to spot it, and five ways to fix the vanishing gradient.

2. Prerequisites

- The [backpropagation what Note](#) and the [backpropagation how Note](#): gradients as chain-rule products.
- The [sigmoid derivative Note](#): $\sigma'(z) = \sigma(z)(1 - \sigma(z)) \leq 0.25$.

3. The vanishing gradient problem

Key point: Three facts: a product of many numbers below 1 is tiny; it only happens in deep networks; and it comes from activations such as sigmoid and tanh whose slopes are small.

3.1 Multiplying small numbers

Key point: $0.1^{10} = 10^{-10}$ and $0.25^{10} \approx 10^{-6}$: the more factors below 1, the smaller the product.

The whole problem rests on one fact of arithmetic. Multiply numbers that are all below 1, and the product is smaller than any of them; the more factors, the smaller it gets. Ten factors of 0.1 give $0.1^{10} = 10^{-10}$.

Two more conditions make this a problem in practice:

- **Depth:** it only appears in **deep neural networks**, with many hidden layers, because only they have long chains of factors.
- **Activation:** it appears with **sigmoid** or **tanh**. Both squash a huge input range into a small output range (0 to 1, or -1 to 1), so their slopes are small.

3.2 Where the small factors come from

Key point: Every sigmoid between a weight and the loss contributes its derivative, at most 0.25, to that weight's gradient.

In the classification network of the [backpropagation how Note](#), the gradient of a first-layer weight was

$$\frac{\partial L}{\partial W_{11}^1} = -(y - \hat{y}) \cdot W_{11}^2 \cdot O_{11}(1 - O_{11}) \cdot x_{i1}$$

The factor $O_{11}(1 - O_{11})$ is the slope of the hidden node's sigmoid. It is never more than 0.25, and close to 0 when the node is saturated near 0 or 1 (see the [sigmoid derivative Note](#)). Each extra sigmoid layer between the weight and the loss adds one more such factor, together with a weight.

1. **In words:** for a weight k sigmoid layers away from the output, the gradient carries k sigmoid slopes, each at most 0.25.

2. **Formula:** with weights near 1, roughly

$$\left| \frac{\partial L}{\partial W} \right| \lesssim |\text{output error}| \times 0.25^k \times |\text{input}|$$

3. **Example:** with $k = 10$ layers, $0.25^{10} = 9.5 \times 10^{-7}$: the gradient is at most about a millionth of what the output layer sees.

3.3 What a tiny gradient does to the update

Key point: A weight of 1 with learning rate 0.01 and gradient 0.0001 becomes 0.999999: no real change, so no learning.

The update is $W_{\text{new}} = W_{\text{old}} - \eta \partial L / \partial W$:

$$W_{\text{new}} = 1 - 0.01 \times 0.0001 = 0.999999$$

The weight has effectively not changed. If the early weights do not change, the loss does not fall either: it stays where it was at the start. The early layers, which are supposed to learn the basic features, learn nothing, and backpropagation cannot converge.

This was one of the main reasons neural networks failed in the 1980s and 1990s, when almost every network used sigmoid or tanh (see section 3.4 of the [types of neural networks Note](#)).

4. Seeing it in Keras

Key point: A network with 10 sigmoid hidden layers on the moons data: first-layer gradients around 10^{-8} , weights unchanged after an epoch, and a loss stuck at 0.70 for 100 epochs.

4.1 The experiment

Key point: 250 points of `make_moons`, 10 hidden layers of 10 sigmoid nodes, plain SGD with learning rate 0.5.

The data is `make_moons` from scikit-learn: 250 points in two interleaved half-moons, two input columns and a class 0 or 1, split 200 for training and 50 for testing. The network has 10 hidden layers of 10 sigmoid nodes and one sigmoid output node, trained with binary cross-entropy.

Python: A deep sigmoid network.

```
import keras
from sklearn.datasets import make_moons

X, y = make_moons(n_samples=250, noise=0.05, random_state=42)
```

```

model = keras.Sequential(
    [keras.Input(shape=(2,))]
    + [keras.layers.Dense(10, activation="sigmoid")
        for _ in range(10)] # 10 hidden layers
    + [keras.layers.Dense(1, activation="sigmoid")])
model.compile(optimizer=keras.optimizers.SGD(learning_rate=0.5),
              loss="binary_crossentropy", metrics=["accuracy"])

```

The list comprehension [... for _ in range(10)] builds 10 identical layers; + joins the lists. Plain SGD makes every update exactly the learning rate times the gradient, so the gradient's size shows directly in the weights.

4.2 The gradient of each layer

Key point: The last layer's gradients average 0.11; the first layer's average 6×10^{-9} : a factor of 17 million.

`tf.GradientTape` gives the exact gradient of the loss with respect to every weight. Averaging the size of the gradients in each layer (Figure 1, red):

Layer	1 (input side)	3	5	7	9	11 (output)
Mean $ \partial L / \partial W $	6.4×10^{-9}	5×10^{-8}	1.9×10^{-6}	6.8×10^{-5}	1.7×10^{-3}	0.11

From the output back to the input, every layer divides the gradient by about 4 to 7, as the sigmoid's slope predicts. On a log scale the red line is almost straight: shrinking by a constant factor per layer.

4.3 The weights do not move, the loss does not fall

Key point: After one epoch, no first-layer weight changed by more than 4×10^{-8} ; after 100 epochs the loss is 0.700 and the test accuracy 46%.

We store the 20 weights of the first layer (2 inputs $\times$ 10 nodes), train for one epoch, and read them again. They agree to six decimals: the largest change among the 20 is 3.7×10^{-8} .

Over 100 epochs (Figure 2, red) the training loss goes from 0.704 to 0.700, next to 0.693, the loss of guessing 0.5 for every point. The test accuracy is 46%, no better than guessing. The first-layer weights moved by 3×10^{-7} on average in all that time.

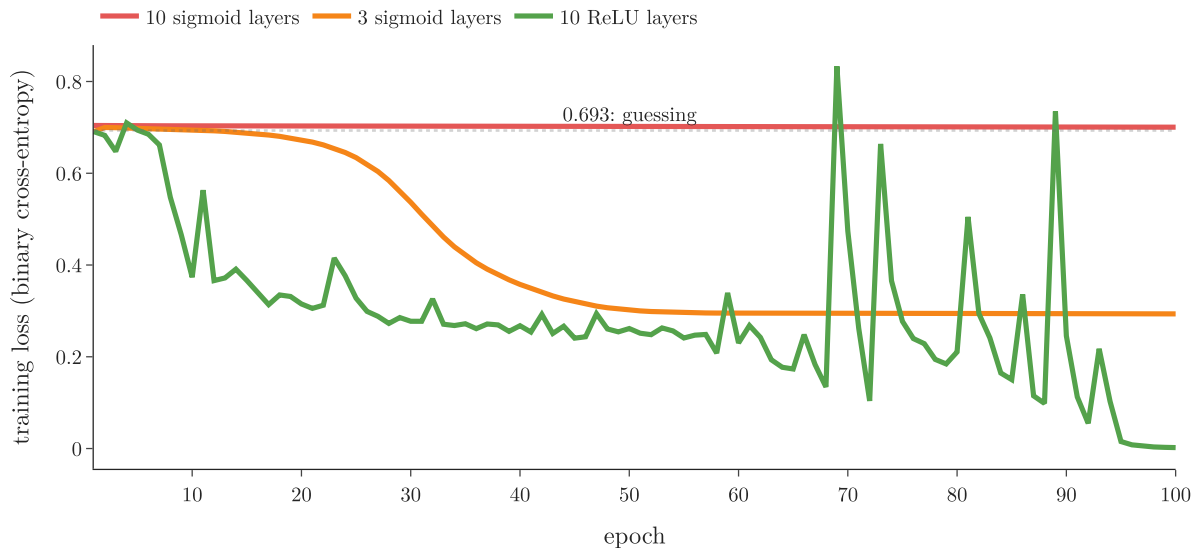


Figure 2: Training loss over 100 epochs: 10 sigmoid layers stay at the guessing level; 3 sigmoid layers and 10 ReLU layers learn

Extra: One way to read a gradient off the weights is $(W_{\text{old}} - W_{\text{new}})/\eta$. This only works for plain SGD and a single update. Keras' default optimizer is Adam, whose steps are scaled per weight and are not proportional to the gradient; there even tiny gradients produce visible weight changes, which hides the problem in the weights (but not in the loss). `tf.GradientTape` gives the true gradient with any optimizer.

5. How to recognise it

Key point: The loss stays flat from epoch to epoch, and the weights of the early layers stay the same.

Two signs:

1. **The loss does not change.** Keras prints the loss after every epoch. If it stays at its starting value, as in Figure 2, the gradients may be vanishing.
2. **The weights do not change.** Plot a weight such as W_{11}^1 against the epoch. A flat line means it is not being updated. Tools such as TensorBoard draw these plots automatically during training.

6. Five ways to fix it

Key point: Fewer layers; ReLU instead of sigmoid; better starting weights; batch normalisation; residual connections. The first two are shown here; the others have their own Notes later.

6.1 Reduce the depth

Key point: With 3 hidden layers instead of 10, the first-layer gradients are only 10 times smaller than the last, and the network learns: loss 0.29, test accuracy 92%.

Fewer layers mean fewer factors in each product. The same experiment with 3 sigmoid hidden layers (Figure 1 and Figure 2, orange):

- the first layer's gradients average 1.3×10^{-3} , only 10 times smaller than the output layer's;
- the loss falls from 0.691 to 0.294 in 100 epochs, and the test accuracy is 92%;
- the first-layer weights moved by 0.59 on average.

This works, but it is rarely the answer. We make networks deep to capture complex patterns; a shallow network may not be able to represent them. Reducing depth is reasonable only when the data is not very complex.

6.2 Use ReLU

Key point: ReLU outputs $\max(0, z)$; its slope is 0 or 1, never a fraction, so it does not shrink the gradient. With 10 ReLU layers the loss reaches 0.002 and the test accuracy 100%.

ReLU (rectified linear unit) is the activation $\text{ReLU}(z) = \max(0, z)$: 0 for negative inputs, the input itself for positive ones. It squashes only the negative side; positive values pass through unchanged.

Its slope is 0 for $z < 0$ and 1 for $z > 0$. A product of 1s stays 1, so the factors on the way back no longer shrink the gradient. Keeping 10 hidden layers but switching them to ReLU (the output stays sigmoid for binary cross-entropy):

- the gradients are about the same size in every layer, between 4×10^{-5} and 3×10^{-4} (Figure 1, green);
- the loss falls from 0.691 to 0.002 in 100 epochs, and the test accuracy is 100% (Figure 2, green);
- the first-layer weights moved by 0.74 on average.

The spikes in the green curve come from the large learning rate of 0.5, chosen so that plain SGD moves at all in the sigmoid networks; ReLU networks usually train with smaller rates.

Python: The only change is the activation.
`keras.layers.Dense(10, activation="relu")`

ReLU has its own weakness, the **dying ReLU**: a node whose input stays negative has slope 0, so its weights get no updates and it stays dead. Variants such as **Leaky ReLU** keep a small slope for negative inputs. Both come with the activation function Notes later.

6.3 Initialise the weights properly

Key point: Starting weights scaled to the size of each layer (Glorot/Xavier, He) keep the backward signal from shrinking or growing.

Random starting weights that are too small make every factor small. Initialisation schemes such as **Glorot (Xavier)** and **He** choose the spread of the random weights from the number of nodes, so that signals keep roughly the same size from layer to layer. They are taught with weight initialisation later.

6.4 Batch normalisation

Key point: A layer that re-scales the values flowing between layers, keeping them in the range where the slopes are large.

Batch normalisation is a type of layer placed between layers of a network. It re-centres and re-scales its inputs during training, which keeps the activations away from the flat ends of sigmoid and tanh. It is taught in its own Note later.

6.5 Residual networks

Key point: Shortcut connections let the gradient skip over layers.

A **residual block** adds a layer's input directly to its output, so the gradient has a path that bypasses the layer's small factors. Networks built from such blocks, the **ResNet** family, are taught with convolutional networks.

7. The exploding gradient problem

Key point: The opposite case: factors above 1 multiply into a huge gradient, the updates become huge, and the loss jumps around or becomes undefined. It is most common in recurrent networks; gradient clipping is the standard fix.

7.1 Multiplying large numbers

Key point: $1.5^{10} = 58$. A gradient of 1000 with learning rate 0.1 moves a weight from 1 to -99 in one step.

If the factors are above 1, the product grows with every one: $1.5^{10} = 58$. Large weights between layers act exactly this way. The update then throws the weight far away:

1. **In words:** a huge gradient times the learning rate gives a huge step.
2. **Formula:** $W_{\text{new}} = W_{\text{old}} - \eta \partial L / \partial W$.
3. **Example:** with $W_{\text{old}} = 1$, $\eta = 0.1$ and $\partial L / \partial W = 1000$:

$$W_{\text{new}} = 1 - 0.1 \times 1000 = -99$$

The next step can be even larger. The weights jump around at random, the loss does not decrease, and the model stops training. This is the **exploding gradient problem**. It shows up most in recurrent neural networks, which multiply by the same weights at every time step; it is studied in detail there.

7.2 An exploding network in Keras

Key point: 10 linear hidden layers with weights of standard deviation 1: gradients of 300 to 2,400 in every layer, and a loss of NaN after the first epoch.

We build 10 hidden layers of 10 nodes again, now with linear activations (no squashing) and starting weights drawn with standard deviation 1. Each layer then roughly triples the signal (about $\sqrt{10}$) in both directions.

- The gradients (Figure 1, right) are between 330 and 2,400 in every layer, against 10^{-4} for the ReLU network.
- Trained with plain SGD and learning rate 0.01, the loss is **nan** (“not a number”) from the first epoch: the numbers overflowed.

7.3 Gradient clipping

Key point: Rescale any gradient whose size exceeds a limit before the update. The same network then trains: loss 12,155 to 2,049 in 5 epochs.

Gradient clipping caps the gradient before the update: if its overall size (its norm) is larger than a chosen limit, it is scaled down to that limit, keeping its direction.

Python: Clipping in Keras.

```
opt = keras.optimizers.SGD(learning_rate=0.01, clipnorm=1.0)
clipnorm=1.0 scales each weight array's gradient down so its norm is at most 1.
```

With clipping the same network produces finite numbers: the loss falls from 12,155 to 2,049 over 5 epochs. Still enormous, because the starting weights are bad, but it decreases instead of breaking. Proper initialisation and batch normalisation also help against exploding gradients.

Extra: The loss can exceed the usual 16 or so because Keras 3 computes binary cross-entropy from the output node's weighted sum (the logit) when the output is a sigmoid. That avoids rounding errors, and it also means the loss of a confidently wrong prediction is no longer capped.

8. Summary

	Vanishing gradient	Exploding gradient
Cause	product of many factors below 1	product of many factors above 1
Typical source	sigmoid or tanh in a deep network	large weights; recurrent networks
Symptom	early weights frozen, loss flat	huge jumps, loss erratic or NaN
Here	first layer 6×10^{-9} vs last 0.11; loss stuck at 0.70	gradients 330 to 2,400; loss NaN
Fixes	fewer layers, ReLU, initialisation, batch normalisation, residual blocks	gradient clipping, initialisation, batch normalisation

- The update is proportional to the gradient; a gradient near 0 means no learning.
- Each sigmoid layer multiplies the gradient by at most 0.25, so early layers in a deep sigmoid network barely learn.
- Watch the loss per epoch, and the early weights, to detect it.
- 3 sigmoid layers or 10 ReLU layers learn the moons data; 10 sigmoid layers cannot.

9. Key terms

Term	Meaning
Deep neural network	A neural network with many hidden layers
Exploding gradient	Gradients growing huge as they pass back through many layers, making updates erratic
ReLU	The activation $\max(0, z)$; slope 0 for negative inputs and 1 for positive ones
Dying ReLU	A ReLU node whose input stays negative, so its slope and updates stay 0
Leaky ReLU	A ReLU variant with a small slope for negative inputs, so nodes do not die
Glorot (Xavier) and He initialisation	Ways to choose the spread of random starting weights from the layer sizes
Batch normalisation	A layer that re-centres and re-scales the values passing between layers during training
Residual block	A block whose input is added to its output, giving the gradient a shortcut
Gradient clipping	Scaling a gradient down to a maximum size before the update

Term	Meaning
<code>make_moons</code>	scikit-learn function that generates two interleaving half-moon classes